

LISTA DE COTEJO PARA EVALUACIÓN PRÁCTICA T1

Sistema CRUD con FastAPI y PostgreSQL

Unidad curricular	Acreditable III
Evaluación	Proyecto práctico grupal - API funcional
Semana de aplicación	Semana 11
Ponderación	15%
Modalidad	Grupal
Producto a entregar	API REST funcional desarrollada con FastAPI, PostgreSQL y documentación automática de FastAPI.

1. Requisitos generales del proyecto

N.º	Criterio a cumplir	Sí	No	Observación
1	El proyecto está desarrollado utilizando FastAPI como framework principal.			
2	El sistema corresponde a la base de datos asignada al grupo.			
3	El proyecto se ejecuta correctamente sin errores críticos.			
4	El sistema tiene una estructura de carpetas organizada y comprensible.			
5	El proyecto cuenta con un archivo principal de ejecución, por ejemplo main.py.			
6	El proyecto permite levantar el servidor con Uvicorn.			
7	El sistema muestra correctamente la documentación automática en /docs.			
8	La API tiene nombres de rutas claros y relacionados con las entidades del sistema.			

2. Diagrama Entidad-Relación de la base de datos

N.º	Criterio a cumplir	Sí	No	Observación
1	El grupo presenta un Diagrama Entidad-Relación correspondiente a la base de datos asignada.			
2	El DER incluye las entidades principales del sistema asignado.			
3	Cada entidad contiene atributos coherentes con la descripción del sistema.			
4	Cada entidad posee una clave primaria correctamente identificada.			
5	El DER muestra las claves foráneas necesarias para relacionar las entidades.			
6	Las relaciones entre entidades están representadas correctamente.			
7	El diagrama indica la cardinalidad de las relaciones, por ejemplo 1:1, 1:N o N:N.			
8	Las relaciones del DER tienen sentido lógico según el funcionamiento real del sistema.			
9	El diseño evita duplicidad innecesaria de datos.			
10	El DER coincide con las tablas implementadas posteriormente en PostgreSQL.			
11	El grupo puede explicar el propósito de cada entidad dentro del sistema.			
12	El grupo puede justificar por qué existen las relaciones planteadas en el diagrama.			

3. Requisitos de conexión con PostgreSQL

N.º	Criterio a cumplir	Sí	No	Observación
1	El proyecto se conecta correctamente a una base de datos PostgreSQL.			
2	La base de datos creada corresponde al sistema asignado al grupo.			
3	Los datos de conexión están definidos de forma clara en el proyecto.			
4	El sistema utiliza un ORM o herramienta de conexión trabajada en clase, como SQLAlchemy o SQLModel.			
5	Las tablas se crean correctamente en PostgreSQL.			
6	Las operaciones realizadas desde la API se reflejan realmente en la base de datos.			
7	El sistema no trabaja únicamente con listas temporales en memoria.			
8	La conexión a la base de datos se encuentra separada del archivo principal cuando la estructura del proyecto lo permita.			

4. Requisitos de modelado de datos

N.º	Criterio a cumplir	Sí	No	Observación
1	El sistema incluye modelos que representan las entidades principales de la base de datos asignada.			
2	Cada modelo posee atributos coherentes con la descripción de la entidad.			
3	Cada tabla cuenta con una clave primaria.			
4	Los tipos de datos utilizados son adecuados para cada campo.			
5	Se aplican campos obligatorios y opcionales de manera lógica.			
6	Se incluyen campos de estado cuando el sistema lo requiere, por ejemplo activo, disponible, pendiente, pagado o cancelado.			
7	Los nombres de los modelos, tablas y campos son claros y representativos.			
8	El diseño de datos responde al funcionamiento lógico del sistema asignado.			

5. Requisitos de validación con Pydantic

N.º	Criterio a cumplir	Sí	No	Observación
1	El sistema utiliza modelos de validación para recibir datos desde las peticiones.			
2	Los datos enviados por el usuario son validados antes de guardarse en la base de datos.			
3	Se evita recibir información innecesaria o peligrosa desde el cuerpo de la petición.			
4	Los esquemas de entrada están separados de los modelos de base de datos cuando corresponde.			
5	Los esquemas de respuesta muestran únicamente la información necesaria.			
6	El sistema responde adecuadamente cuando faltan datos obligatorios.			
7	Se utilizan tipos de datos correctos como str, int, float, bool, date, entre otros.			
8	La validación contribuye a mantener ordenada y segura la información del sistema.			

6. Operaciones CRUD básicas

N.º	Criterio a cumplir	Sí	No	Observación
1	El sistema permite crear registros mediante endpoints POST.			
2	El sistema permite consultar todos los registros mediante endpoints GET.			
3	El sistema permite consultar un registro específico por su identificador.			
4	El sistema permite actualizar registros mediante PUT o PATCH.			
5	El sistema permite eliminar registros mediante DELETE o aplicar eliminación lógica cuando corresponda.			
6	Las rutas CRUD están implementadas para las entidades principales del sistema.			
7	Las operaciones CRUD modifican realmente la información en PostgreSQL.			
8	Los endpoints devuelven respuestas claras después de cada operación.			
9	No existen rutas duplicadas o mal nombradas que generen confusión.			
10	El CRUD responde a la lógica real del sistema asignado.			

7. CRUD funcional ampliado y acciones lógicas del sistema

N.º	Criterio a cumplir	Sí	No	Observación
1	El sistema incluye búsquedas lógicas según su naturaleza, por ejemplo por SKU, documento, placa, correo, código, número de reserva, número de factura o número de guía.			
2	El sistema permite filtrar información según criterios útiles, como estado, categoría, fecha, disponibilidad, cliente, usuario o entidad relacionada.			
3	El sistema incluye endpoints de consulta que serían necesarios si existiera una futura interfaz web.			
4	El sistema permite consultar registros asociados a otra entidad, por ejemplo citas de un paciente, productos de una categoría, pagos de una reserva o ventas de un cliente.			
5	El sistema permite actualizar campos específicos importantes, por ejemplo stock, estado, precio, disponibilidad, fecha, monto o datos de contacto.			
6	Las rutas implementadas no son genéricas únicamente, sino que responden a necesidades reales del sistema asignado.			
7	El grupo puede explicar por qué cada endpoint lógico es necesario dentro del funcionamiento del sistema.			
8	Las acciones ampliadas mantienen coherencia con el CRUD: crear, leer, actualizar o eliminar información de forma lógica.			

8. Requisitos de relaciones entre entidades

N.º	Criterio a cumplir	Sí	No	Observación
1	El sistema incluye relaciones entre las entidades principales.			
2	Se utilizan claves foráneas cuando una entidad depende de otra.			
3	Las relaciones uno a muchos están implementadas correctamente cuando aplican.			
4	Las relaciones muchos a muchos están implementadas cuando el sistema lo requiere.			
5	Las rutas permiten trabajar con datos relacionados de forma lógica.			
6	No se crean registros relacionados con identificadores inexistentes.			
7	La estructura de la base de datos evita duplicidad innecesaria de información.			
8	Las relaciones se pueden explicar claramente durante la presentación técnica.			

9. Requisitos de manejo básico de errores

N.º	Criterio a cumplir	Sí	No	Observación
1	El sistema devuelve un error adecuado cuando se busca un registro inexistente.			
2	Se utiliza HTTPException cuando corresponde.			
3	Los mensajes de error son claros y comprensibles.			
4	El sistema no se detiene completamente ante errores comunes del usuario.			
5	Se validan casos como identificadores inexistentes, datos incompletos o registros no encontrados.			
6	Los códigos de estado HTTP utilizados son coherentes con la situación.			
7	Las respuestas de error ayudan a comprender qué ocurrió.			
8	El manejo de errores está aplicado en las operaciones principales del CRUD.			

10. Requisitos de documentación automática con FastAPI

N.º	Criterio a cumplir	Sí	No	Observación
1	La API muestra correctamente la documentación en /docs.			
2	Las rutas aparecen organizadas y son comprensibles en Swagger UI.			
3	Los nombres de los endpoints permiten identificar su función.			
4	Los modelos de entrada y respuesta se visualizan correctamente en la documentación.			
5	La documentación permite probar las operaciones CRUD desde el navegador.			
6	Los endpoints tienen nombres, parámetros y cuerpos de solicitud claros.			
7	La documentación refleja el funcionamiento real del sistema.			

11. Requisitos de organización del código

N.º	Criterio a cumplir	Sí	No	Observación
1	El código está organizado en archivos o carpetas según su función.			
2	El archivo principal no concentra toda la lógica innecesariamente.			
3	Existe separación entre modelos, conexión a base de datos, rutas y esquemas cuando corresponde.			
4	Los nombres de archivos y carpetas son claros.			
5	El código es legible y mantiene una estructura ordenada.			
6	Se evita repetir código innecesariamente.			
7	Se utilizan funciones o dependencias para reutilizar lógica cuando sea necesario.			
8	El proyecto puede ser entendido por otra persona al revisar su estructura.			

12. Requisitos mínimos de presentación técnica

N.º	Criterio a cumplir	Sí	No	Observación
1	El grupo explica cuál es el propósito del sistema desarrollado.			
2	El grupo presenta las entidades principales de su base de datos.			
3	El grupo explica la relación entre las entidades.			
4	El grupo demuestra la conexión con PostgreSQL.			
5	El grupo ejecuta la API durante la presentación.			
6	El grupo prueba operaciones CRUD desde Swagger UI o herramienta equivalente.			
7	El grupo evidencia que los datos se guardan, consultan, actualizan y eliminan en la base de datos.			
8	Cada integrante participa en la explicación técnica.			
9	El grupo responde preguntas básicas sobre el funcionamiento del sistema.			
10	La exposición demuestra dominio general del proyecto desarrollado.			

Condiciones mínimas para considerar el sistema como entregado

1. Debe ejecutarse correctamente en FastAPI.
2. Debe conectarse a PostgreSQL.
3. Debe estar basado en la base de datos asignada al grupo.
4. Debe presentar el Diagrama Entidad-Relación de la base de datos.
5. Debe tener modelos de datos coherentes con las entidades del sistema.
6. Debe implementar operaciones CRUD funcionales básicas y lógicas.
7. Debe tener relaciones reales entre entidades.
8. Debe mostrar la documentación automática en /docs.
9. Debe permitir demostrar el funcionamiento durante la presentación técnica.

Aclaratoria sobre el CRUD funcional ampliado

El CRUD no debe limitarse únicamente a crear, listar, actualizar y eliminar registros de forma general. Cada grupo debe analizar cómo funcionaría su sistema si tuviera una interfaz web y fuera utilizado por un usuario real. Por ejemplo, en un sistema de productos debe existir la posibilidad de buscar por SKU, filtrar por categoría, consultar stock o actualizar disponibilidad. En un sistema de citas, debe ser posible consultar citas por paciente, médico, fecha o estado. Todas esas operaciones siguen formando parte del CRUD, porque crean, leen, actualizan o eliminan información, pero aplicadas a la lógica real del sistema.

El Diagrama Entidad-Relación no debe verse como un elemento separado del sistema. Las entidades, atributos y relaciones representadas en el diagrama deben coincidir con las tablas creadas en PostgreSQL y con los modelos implementados en FastAPI. Si el DER muestra una relación entre dos entidades, esa relación debe reflejarse en la base de datos mediante claves foráneas o tablas intermedias cuando corresponda.